

Principles of Object-Oriented Design

 [_ \(https://github.com/learn-co-curriculum/python-p3-oop-design-principles\)](https://github.com/learn-co-curriculum/python-p3-oop-design-principles)  [_ \(https://github.com/learn-co-curriculum/python-p3-oop-design-principles/issues/new\)](https://github.com/learn-co-curriculum/python-p3-oop-design-principles/issues/new)

Learning Goals

- Define "design" in the context of coding.
 - Explain why design principles are important for you as a programmer.
 - Learn some of the basic design principles that object-oriented programmers employ.
-

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Initialize**: create a working copy of a class using its `__init__()` method.
 - **Instance**: one specific working copy of a class. It is created when a class's `__init__()` method is called.
 - **Object**: the more common name for an instance. The two can usually be used interchangeably.
 - **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
 - **Procedural Programming**: programming that is oriented around procedures that are called in sequential order. Fortran and C are procedural programming languages.
 - **Function**: a series of steps that create, transform, and move data.
 - **Method**: a function that is defined inside of a class.
 - **Magic Method**: a special type of method in Python that starts and ends with double underscores. These methods are called on objects under certain conditions without needing to use their names explicitly. Also called **dunder methods** (for **double underscore**).
 - **Attribute**: variables that belong to an object.
 - **Property**: attributes that are controlled by methods.
-

Introduction

A program's *design* refers to the manner in which you, the programmer, organize and arrange its code. The manner in which you arrange your code may seem incidental to the overarching objective of writing code that *works*, of creating a program that behaves as you want and need it to. Implementing design principles, however, is not incidental to writing working code — it is the manner in which you will ensure that the code you write will continue to work in the future.

As programmers, we must code for the future. What does that mean? It means that, inevitably, our programs will grow and require change.

Let's say you wrote a super cool web application that becomes wildly popular — how will your program change to accommodate more users and more traffic and the needs of those users? On the other hand, let's say you build an amazing application for a client who then changes their mind about an important feature. What do you do? Scrap all of your work and begin again? By writing code that is flexible and accommodates change, you won't have to.

As programmers, we combine the functions of inventors and artists, and the code we write should be designed to function well; be flexible to accommodate future change; and be organized in a sensible, logical, even beautiful way.

Basic Design Principles

The study and refinement of the principles of object-oriented design will take place over decades of a programmer's life and career. This reading is meant to function as a very basic introduction to the *what* and *why* of object-oriented design. Here, we'll briefly discuss some of the basic principles. Keep these principles in mind as you continue to learn more about object-oriented Python and revisit them in your future work.

The Single Responsibility Principle and Separation of Concerns

The **single responsibility principle** is the idea that classes in object-oriented programming should have **one job**, one responsibility, and their services (i.e., methods) should be aligned with that responsibility. This principle goes hand in hand with the **separation of concerns**: the idea that the various responsibilities, or *concerns*, of a computer program should be separated out into **discrete sections**.

Let's take the example of an online shopping web application. Such an application has a number of jobs to handle: we need to have users that sign in and purchase items; we have the items themselves that we are selling; we likely have a shopping cart as well. We could develop an

application that takes care of all of these jobs in the following manner:

```
class MyStore:

    def sign_in(self, user):
        self.user = user

    def current_user(self):
        return self.user

    def item(self, item):
        self.item = item

    def item_price(self, price):
        self.item_price = price

    def shopping_cart(self):
        self.shopping_cart = list()

    def add_item_to_cart(self, item):
        self.shopping_cart.append(item)
```

As you can see, our code starts to get out of control relatively quickly. What happens when our program needs to grow to accommodate a feature like coupons? Will we continue to add code to this one file? What happens if our program breaks? How will we determine which of our many methods is responsible for the bug? What if we need to change a particular feature of our app? How can we be sure that we've tracked down all of the affected methods?

Instead, we want to **separate the concerns**, or responsibilities, of such an application, wrapping each concern in its own class that produces its own objects.

We could write a `User` class, an `Item` class and a `ShoppingCart` class. The `User` class should be responsible for assigning a user a name and other details and signing a user in and out. The `Item` class should have methods that describe an item's attributes, including its price. The

`ShoppingCart` class can collect individual item objects and total their price at the time of checkout.

By creating classes with their own responsibilities, we create an application that is organized and logical, and accommodates future change. We also create a program that doesn't give us a headache to even look at.

Abstraction and Don't Repeat Yourself (DRY)

Another basic OO principle is that of **Don't Repeat Yourself** (DRY). If you find yourself repeating the same lines of code again and again, that is a good candidate for refactoring. As programmers, we are lazy (in a good way!) — we are always looking for ways to achieve the desired behavior by writing less code, and we hate to repeat ourselves. Let's take a look at an example by revisiting our online shopping application.

Lately, our client has been offering a variety of discounts. At the time of checkout, a user can tell our application that they have a 10, 25, or 50 percent discount. So, our program needs to determine if a user has a discount and then apply it, if present. Let's take a look at the `checkout()` method of our `ShoppingCart` class.

```
def checkout(self, discount=0):
    # The shopping_cart method returns an array of all the user's items
    total = sum([item.price for item in self.shopping_cart()])

    if discount == 10:
        total -= total * 10 / 100.00
    elif discount == 20:
        total -= total * 20 / 100.00
    elif discount == 50:
        total -= total * 50 / 100.00

    return total
```

In the above method, we are using an `if/else` statement to determine what kind of discount a user has, if any, and then doing some math to calculate the appropriate total. In order to take a discount into account, we are using six lines of code every time we check out a user! That's a lot of repetition. Additionally, what if our application needs to apply coupons to a user's total *before* they check out? Maybe our client has decided to

give their users 20% on their current total if they refer a friend to the site. We would have to use the same `if/else` statement elsewhere in our code. That's a lot of code to constantly re-type.

Top-Tip: If you find yourself copy/pasting the same lines of code more than three times in order to re-use it throughout your program, consider it a "code smell". A code smell is a sign that you should refactor.

Let's refactor this by creating an attribute to access data about a coupon amount. Then, our `checkout()` method can simply use the `coupon` attribute to help it calculate the user's total:

```
class ShoppingCart:
    def __init__(self, coupon=0):
        self.coupon = coupon

    def checkout(self):
        total = sum([item.price for item in self.shopping_cart()])
        total -= total * self.coupon / 100
        return total
```

We've achieved the same behavior with less code. This refactor has the added benefit of being *flexible*, meaning it will accommodate future change. If our client decides to add a new type of coupon, a 23% off discount, the coupon method, having *abstracted* away the value of the coupon, will still work. This is the benefit of abstraction, and we prefer abstracting out attributes by wrapping them inside methods over having our code rely on literal values.

Domain Modeling

One important concept to help organize your object oriented code is the idea of **domain modeling**.

A domain model is a representation of real-world concepts in software. The concept of domain modeling is key in object orientation. In object orientation, we think of our classes as templates for objects. The instances of our classes are thought of as objects. For example, a `Person` class produces people objects that have attributes (data) and behaviors, as described and enacted by instance methods.

Let's think of an example on a slightly larger scale. Let's say we are writing an application that will be used by a major automobile manufacturer to help manage their plants. In this case, we need a program that represents not just individual cars, but the entire car factory. In this (simplified) example, you could write an `AutoPlant` class that produces individual cars and has instance methods that handle things like `take_inventory` or `paint_cars`.

Such a program not only represents a single object or concept but an entire environment — that of the auto factory. Consequently, this program could be understood as a **domain model**. It takes the world of the auto factory, and maps the individual parts of that world to different parts of your program.

As we begin to write more and more advanced programs, you'll see that to really accurately model a domain, we'll need to build a program that contains more than one class.

Conclusion

This has been a very brief intro into the what, why and how of design principles in object-oriented Python. These principles are meant to guide you, not limit you. These principles are also just the very tip of the OO design iceberg. You'll learn more about design over the remainder of this course and you'll learn even more about design over the course of the rest of your programming life.

Resources

- [Object-Oriented Programmin \(OOP\) in Python 3 - Real Python](https://realpython.com/python3-object-oriented-programming/) ➞ [\(https://realpython.com/python3-object-oriented-programming/\)](https://realpython.com/python3-object-oriented-programming/)